Naïve estimates of occupancy and abundance

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-20

Table of contents

Preamble 1

Nuisance variables 2

Offsets . . . 8

Distance effects . . . 8

Duration effect . . . 10

Accounting for different field protocols . . . 15

Preamble

```
suppressPackageStartupMessages({
    library(dplyr)
    library(mefa4)
    library(detect)
    library(ggplot2)
    theme_set(theme_light())
    library(visreg)
})
```

Assemble the ABMI data:

```
if (!interactive()) {
    d <- readRDS("../data/ab_birds_site_year.rds")
} else {
```

```r
    d <- readRDS("data/ab_birds_site_year.rds")
}

spp <- "yellowrumped_warbler"

x <- data.frame(y = d$counts[, spp], d$vars)

x$yday <- as.POSIXlt(x$fielddate)$yday / 365
x$hr <- x$fieldtime / 24
x <- x |>
    dplyr::filter(
        hr > 0.1,
        hr < 0.6,
        yday > 0.37,
        yday < 0.5
    )
```

Nuisance variables

Nuisance variables are covariates whose effects we want to control but we are not really interested in their effect. We just need to estimate them so that we can account for them as best as we can.

We have 3 such covariates in our example:

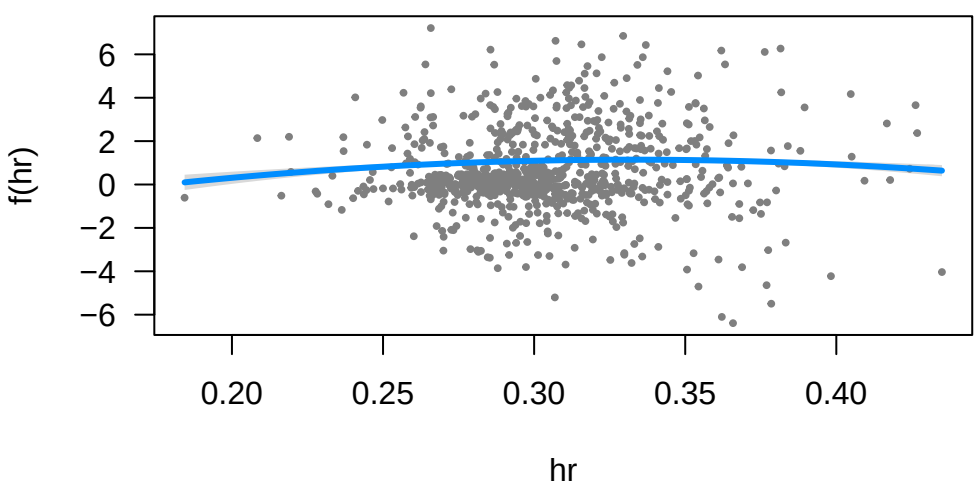
- `hr`: survey time, decimal 24 hr clock / 24
- `yday`: day of the year, ordinal day / 365
- `wind`: wind speed using Beaufort scale

```r
m1 <- glm(
    y ~
        pconif + I(pconif^2) + ahm + thf +
        hr + I(hr^2) + yday + wind,
    data = x, family = poisson
)
summary(m1)
```

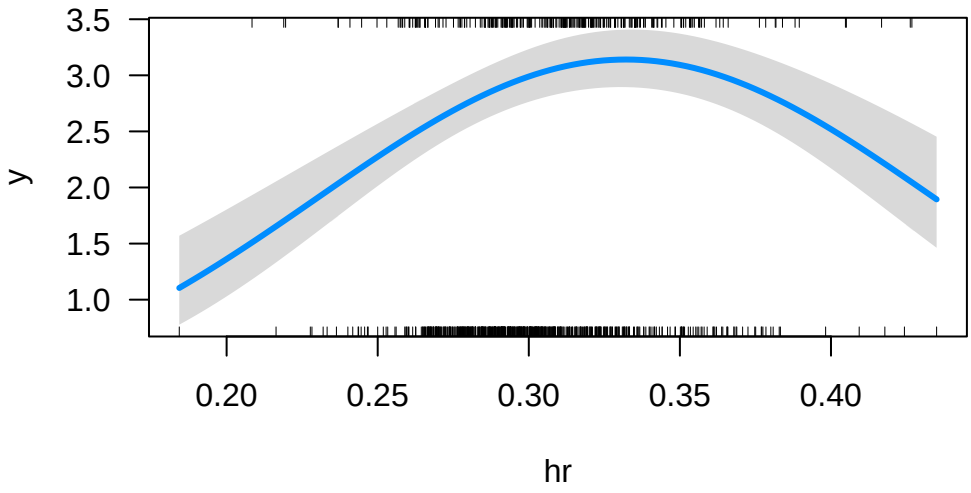
```
Call:
glm(formula = y ~ pconif + I(pconif^2) + ahm + thf + hr + I(hr^2) +
    yday + wind, family = poisson, data = x)
```

```
Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.483291   1.068490  -0.452    0.651
pconif        3.016012   0.204051  14.781  < 2e-16 ***
I(pconif^2)  -2.186466   0.188401 -11.605  < 2e-16 ***
ahm          -0.081051   0.004308 -18.813  < 2e-16 ***
thf          -1.051043   0.090342 -11.634  < 2e-16 ***
hr           31.782253   5.943834   5.347 8.94e-08 ***
I(hr^2)     -47.844913   9.218882  -5.190 2.10e-07 ***
yday         -3.888517   0.794925  -4.892 1.00e-06 ***
wind         -0.088767   0.018362  -4.834 1.34e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 3133.3  on 825  degrees of freedom
AIC: 4774.6

Number of Fisher Scoring iterations: 6
```

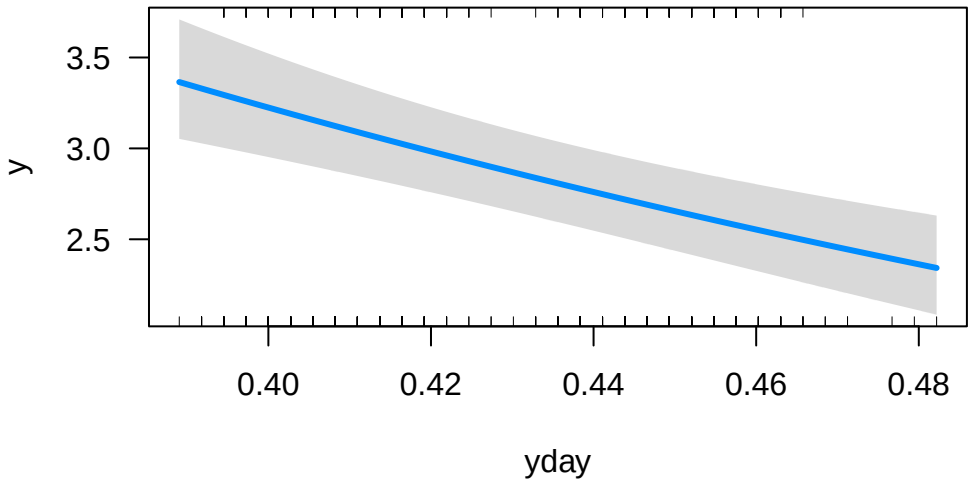
```r
visreg::visreg(m1, "hr", scale = "linear") # quadratic
```



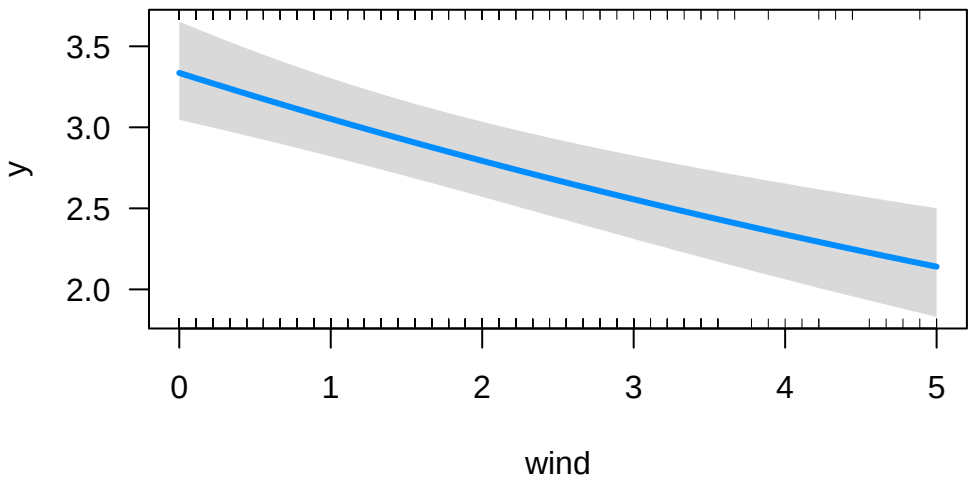

```r
visreg::visreg(m1, "hr", scale = "response") # back-transformed to response scale
```




```
visreg::visreg(m1, "yday", scale = "response")
```




```
visreg::visreg(m1, "wind", scale = "response")
```

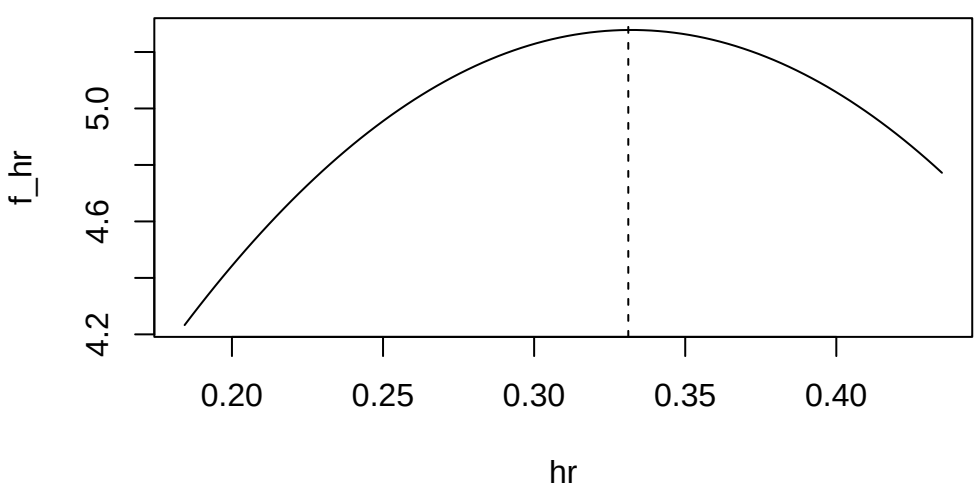


When predicting abundance and accounting for nuisance variables, we want to set the values that maximize the prediction. Two nuisance variables have a negative effect, so we need to pick the min:

```
xnew <- x
xnew$yday <- min(xnew$yday)
xnew$wind <- min(xnew$wind)
```

The quadratic curve for `hr` means that we need to find the value of `hr` that maximizes the partial effect:

```
hr <- seq(min(x$hr), max(x$hr), length.out = 100)
f_hr <- coef(m1)["hr"] * hr + coef(m1)["I(hr^2)"] * hr^2
plot(hr, f_hr, type = "l")
abline(v = hr[which.max(f_hr)], lty = 2)
```



```
xnew$hr <- hr[which.max(f_hr)]
```

`Pred` is the predicted abundance for the original data (same as `fitted(m1)`), while `PredNew` is the predicted abundance for the new data with optimized nuisance variables. The mean predicted abundance is higher when we optimize the nuisance variables:

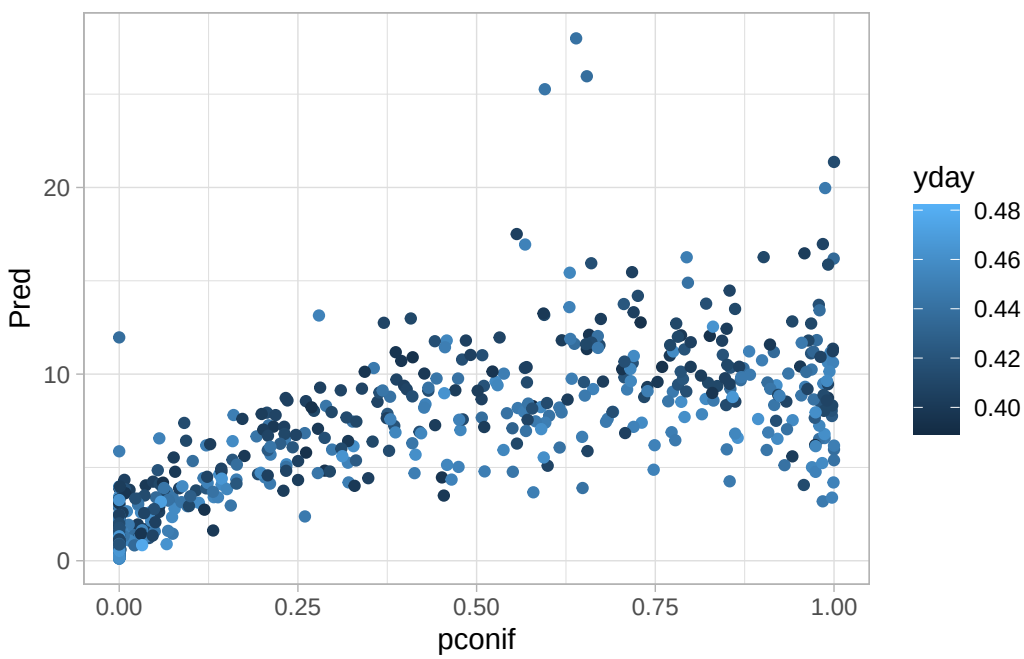
```
x$Pred <- predict(m1, x, type = "response")
x$PredNew <- predict(m1, xnew, type = "response")
mean(x$Pred)
```

```
[1] 4.539568
```

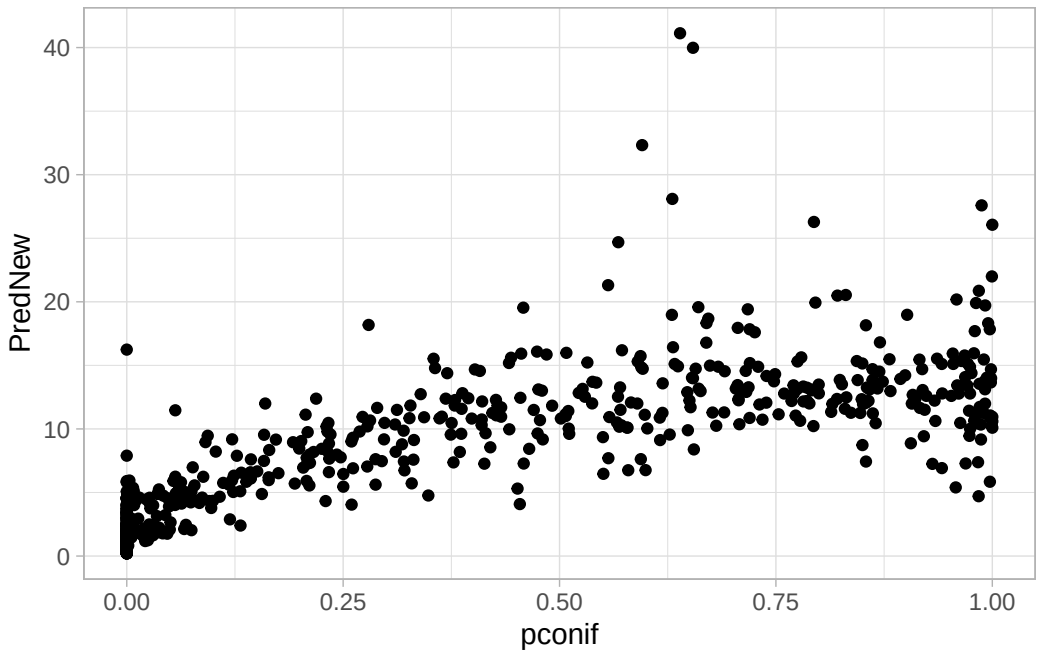
```
mean(x$PredNew)
```

```
[1] 6.293655
```

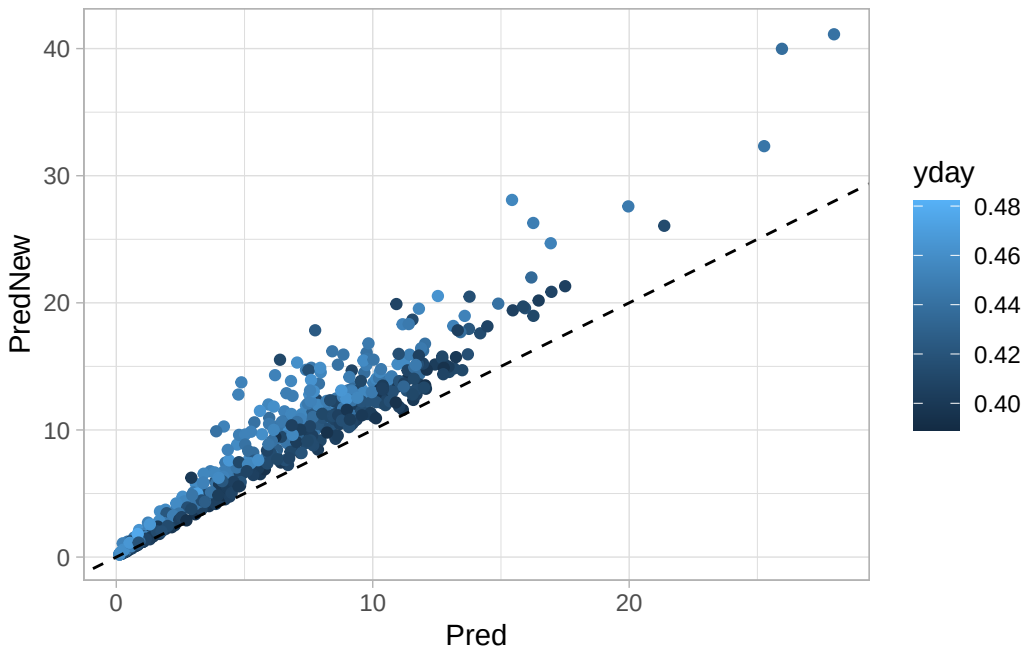
```
x |> ggplot(aes(x = pconif, y = Pred, col = yday)) +
    geom_point() +
    theme_light()
```



```r
x |> ggplot(aes(x = pconif, y = PredNew)) +
    geom_point() +
    theme_light()
```




```r
x |> ggplot(aes(x = Pred, y = PredNew, col = yday)) +
    geom_point() +
    geom_abline(intercept = 0, slope = 1, lty = 2) +
    theme_light()
```



Offsets

Offsets are constant terms in the linear predictor, e.g. $log(\lambda_i) = \beta_0 + \beta_1 x_{1i} + o_i$, where o_i is an offset.

In the survey area case, an offset might be the log of area surveyed. Abundance (N) is population density (D) multiplied by survey area (A). The logic for this is based on point processes: intensity is a linear function of area under a homogeneous Poisson point process. So we can say that $E[Y_i] = N_i = D_i A_i$, thus $log(N_i) = log(D_i) + log(A_i)$ where $o_i = log(A_i)$ is the offset.

The problem is that we do not know the survey area. We used 10 minutes and unlimited distance counts.

Let's see if using area as offset makes our models comparable. Instead of mixing up different survey types, let's see if we can make them identical. We use distance in meters divided by 100, so the population density is estimated in ha.

Distance effects

We cannot use the unlimited distance counts to evaluate the effect of different point count radii on our estimates. See the JOSM data example for this.

We will use the 150 m radius and assume that distance effects are negligible within this radius. We will use the log of the area of a circle with 150 m radius as an offset:

```
x$logA_150m <- log(pi * (150 / 100)^2)

m2 <- update(m1, y ~ . + offset(logA_150m))
summary(m2)
```

```
Call:
glm(formula = y ~ pconif + I(pconif^2) + ahm + thf + hr + I(hr^2) +
    yday + wind + offset(logA_150m), family = poisson, data = x)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.438951   1.068490  -2.283   0.0225 *
pconif        3.016012   0.204051  14.781  < 2e-16 ***
I(pconif^2)  -2.186466   0.188401 -11.605  < 2e-16 ***
ahm          -0.081051   0.004308 -18.813  < 2e-16 ***
thf          -1.051043   0.090342 -11.634  < 2e-16 ***
hr           31.782253   5.943834   5.347 8.94e-08 ***
I(hr^2)     -47.844913   9.218882  -5.190 2.10e-07 ***
yday         -3.888517   0.794925  -4.892 1.00e-06 ***
wind         -0.088767   0.018362  -4.834 1.34e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6717.5  on 833  degrees of freedom
Residual deviance: 3133.3  on 825  degrees of freedom
AIC: 4774.6

Number of Fisher Scoring iterations: 6
```

We use units of 100 m to get density estimates in ha. Notice that the intercept is now much lower. This is because the offset accounts for the area surveyed, so the model is now estimating density (birds per ha) instead of abundance (birds per point count):

```
coef(m1)
```

```
 (Intercept)       pconif  I(pconif^2)          ahm          thf           hr
 -0.48329135   3.01601181  -2.18646555  -0.08105077  -1.05104331  31.78225295
     I(hr^2)         yday         wind
-47.84491274  -3.88851713  -0.08876712
```

```r
coef(m2)
```

```
 (Intercept)       pconif  I(pconif^2)          ahm          thf           hr
 -2.43895145   3.01601181  -2.18646555  -0.08105077  -1.05104331  31.78225295
     I(hr^2)         yday         wind
-47.84491274  -3.88851713  -0.08876712
```

```r
coef(m1)[1] - coef(m2)[1] # log of area ratio
```

```
(Intercept)
    1.95566
```

```r
log(pi * (150 / 100)^2)
```

```
[1] 1.95566
```

When the offset varies from site to site, the effects are more complex, but the principle is the same.

Duration effect

Next we will inspect what effect the count duration has on our estimates. First we need to get the counts corresponding to the 3 time intervals:

- 0–3.3 mins
- 0–6.6 mins
- 0–10 mins

```r
if (!interactive()) {
    dp <- readRDS("../data/ab_birds_site_year_pt.rds")
} else {
    dp <- readRDS("data/ab_birds_site_year_pt.rds")
}
```

```r
y123 <- mefa4::groupSums(
    cbind(
        y1 = dp$removal$int1[, spp],
        y2 = dp$removal$int2[, spp],
        y3 = dp$removal$int3[, spp]
    ),
    1,
    dp$vars$site_year
)
x <- data.frame(y123[rownames(x), ], x)

# cumulative counts
x$y12 <- x$y1 + x$y2
```

The models for the 1st and the 1st + 2nd time intervals are:

```r
m3 <- update(m2, y1 ~ .)
summary(m3)
```

```
Call:
glm(formula = y1 ~ pconif + I(pconif^2) + ahm + thf + hr + I(hr^2) +
    yday + wind + offset(logA_150m), family = poisson, data = x)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.649256   1.253310  -2.114   0.0345 *
pconif        3.049172   0.238913  12.763  < 2e-16 ***
I(pconif^2)  -2.201645   0.220181  -9.999  < 2e-16 ***
ahm          -0.082177   0.005065 -16.224  < 2e-16 ***
thf          -1.119981   0.107440 -10.424  < 2e-16 ***
hr           32.909861   6.983188   4.713 2.44e-06 ***
I(hr^2)     -49.911415  10.837915  -4.605 4.12e-06 ***
yday         -4.431317   0.930013  -4.765 1.89e-06 ***
wind         -0.087358   0.021456  -4.071 4.67e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 5257.7  on 833  degrees of freedom
```

```
Residual deviance: 2556.1  on 825  degrees of freedom
AIC: 4021.8

Number of Fisher Scoring iterations: 6
```

```r
m4 <- update(m2, y12 ~ .)
summary(m4)
```

```
Call:
glm(formula = y12 ~ pconif + I(pconif^2) + ahm + thf + hr + I(hr^2) +
    yday + wind + offset(logA_150m), family = poisson, data = x)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.638480   1.143330  -2.308    0.021 *
pconif        2.994367   0.216567  13.827  < 2e-16 ***
I(pconif^2)  -2.161640   0.199913 -10.813  < 2e-16 ***
ahm          -0.080162   0.004574 -17.524  < 2e-16 ***
thf          -1.105502   0.096619 -11.442  < 2e-16 ***
hr           33.968104   6.374065   5.329 9.87e-08 ***
I(hr^2)     -51.337887   9.889442  -5.191 2.09e-07 ***
yday         -4.520858   0.844381  -5.354 8.60e-08 ***
wind         -0.086127   0.019467  -4.424 9.68e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6138.3  on 833  degrees of freedom
Residual deviance: 2932.3  on 825  degrees of freedom
AIC: 4501.6

Number of Fisher Scoring iterations: 6
```

Predictions using `xnew2`, setting the log area offset to 0 (unit area of 1 ha) to get density estimates:

```r
xnew2 <- xnew
xnew2$logA_150m <- 0
```

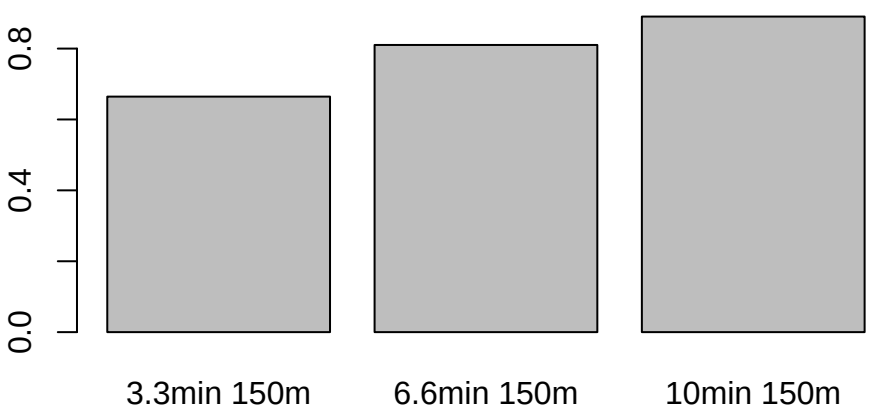
```r
xnew2$Pred_3.3min <- predict(m3, xnew2, type = "response")
xnew2$Pred_6.6min <- predict(m4, xnew2, type = "response")
xnew2$Pred_10min <- predict(m2, xnew2, type = "response")
```

The 10 minutes estimates are higher due to accumulating more detections over the longer duration:

```r
D_est <- c(
    "3.3min 150m" = mean(xnew2$Pred_3.3min),
    "6.6min 150m" = mean(xnew2$Pred_6.6min),
    "10min 150m"  = mean(xnew2$Pred_10min)
)
D_est
```

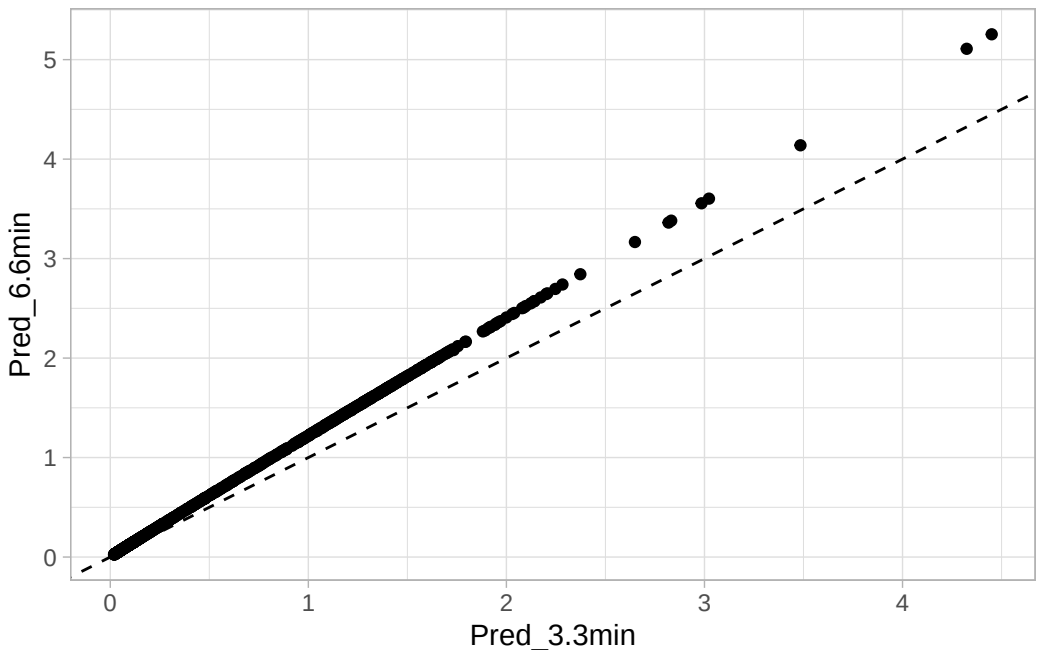
```
3.3min 150m 6.6min 150m  10min 150m
  0.6645231   0.8102462   0.8903701
```

```r
barplot(D_est)
```

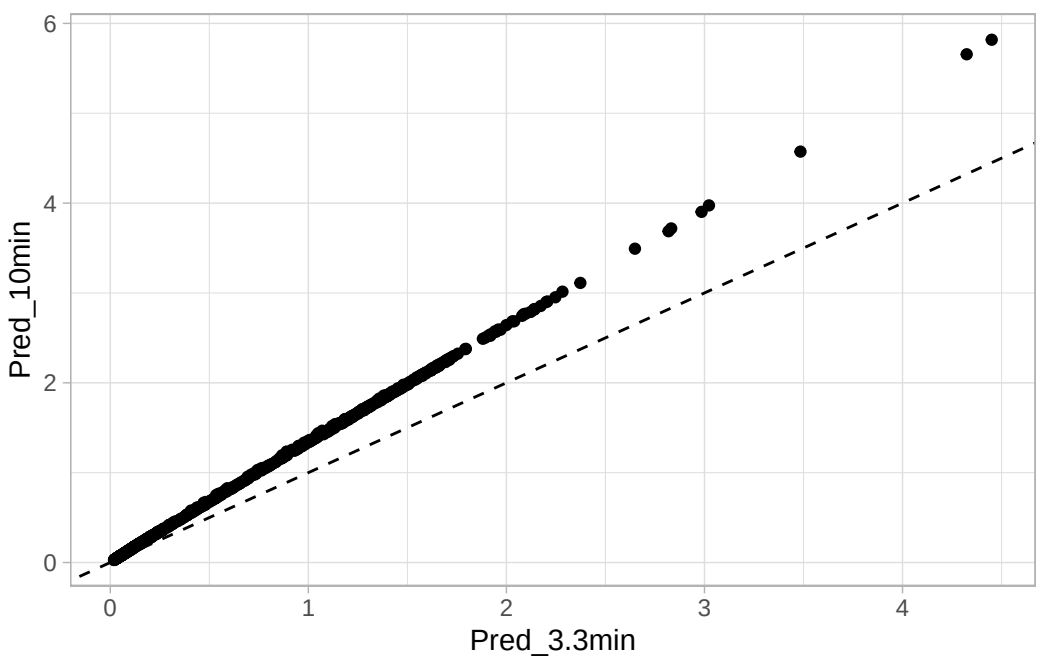


Make some plots to compare the site level predictions:

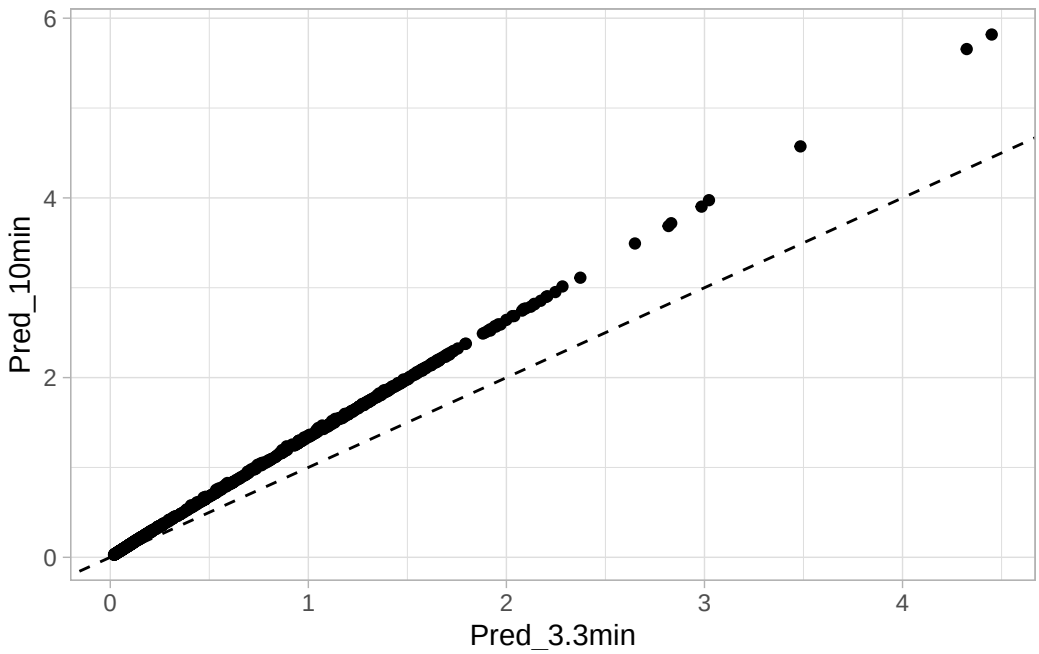
```r
xnew2 |> ggplot(aes(x = Pred_3.3min, y = Pred_6.6min)) +
    geom_point() +
    geom_abline(intercept = 0, slope = 1, lty = 2)
```




```
xnew2 |> ggplot(aes(x = Pred_3.3min, y = Pred_10min)) +
    geom_point() +
    geom_abline(intercept = 0, slope = 1, lty = 2)
```



```r
xnew2 |> ggplot(aes(x = Pred_3.3min, y = Pred_10min)) +
    geom_point() +
    geom_abline(intercept = 0, slope = 1, lty = 2)
```



Accounting for different field protocols

When we survey some areas, the survey protocol is the same for all sites. However, when we combine different datasets, we might find that each might have a slightly different sampling design.

Some surveys might be 10 min, other might be 0–3.3 min, etc. We will assign one of 3 survey protocols to each site, and set the `Count` values corresponding to the time and distance intervals.

```r
Methods <- c("3.3min", "6.6min", "10min")
x$Method <- factor(sample(
    Methods,
    nrow(x),
    replace = TRUE
), levels = Methods)
x$y_Rnd <- x$y1
x$y_Rnd[x$Method == "6.6min"] <- x$y12[x$Method == "6.6min"]
x$y_Rnd[x$Method == "10min"] <- x$y[x$Method == "10min"]
```

This next model will have `y_Rnd` as the response, and we add the `Method` variable and the offset `offset(logA_Rnd)`:

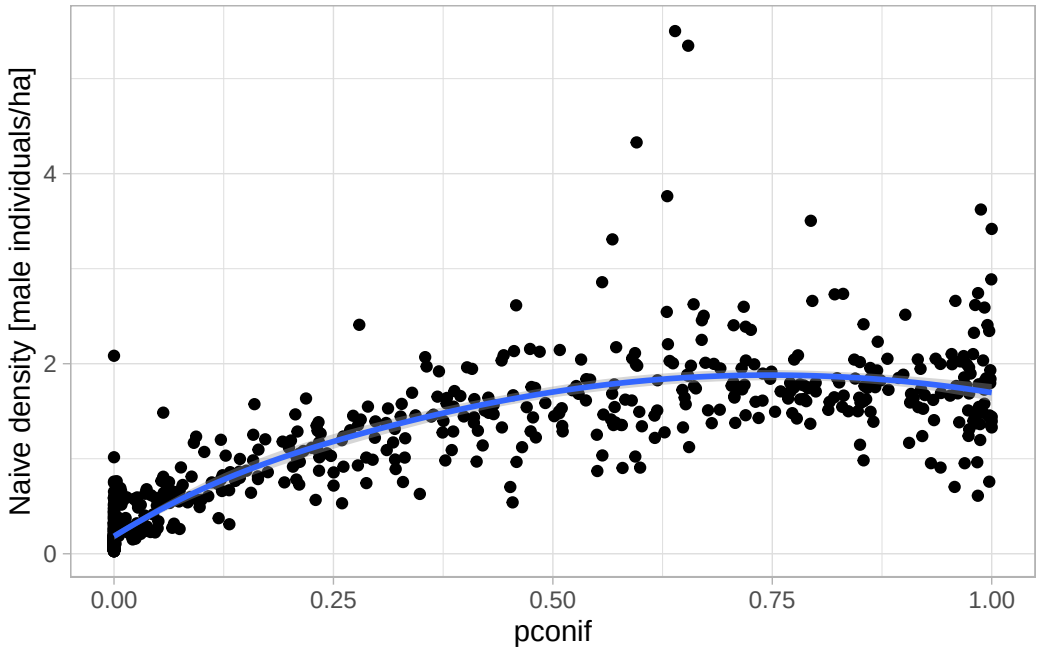
```
m5 <- update(m2, y_Rnd ~ . + Method)
summary(m5)
```

```
Call:
glm(formula = y_Rnd ~ pconif + I(pconif^2) + ahm + thf + hr +
    I(hr^2) + yday + wind + Method + offset(logA_150m), family = poisson,
    data = x)

Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)   -2.881722   1.167827  -2.468   0.0136 *
pconif         3.158087   0.218695  14.441  < 2e-16 ***
I(pconif^2)   -2.307678   0.202017 -11.423  < 2e-16 ***
ahm           -0.080725   0.004643 -17.386  < 2e-16 ***
thf           -1.067247   0.097846 -10.907  < 2e-16 ***
hr            32.635771   6.485533   5.032 4.85e-07 ***
I(hr^2)      -49.430759  10.013230  -4.937 7.95e-07 ***
yday          -4.193376   0.848920  -4.940 7.83e-07 ***
wind          -0.075431   0.019368  -3.895 9.84e-05 ***
Method6.6min   0.443902   0.044980   9.869  < 2e-16 ***
Method10min    0.352809   0.045474   7.758 8.60e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 6096.3  on 833  degrees of freedom
Residual deviance: 2821.8  on 823  degrees of freedom
AIC: 4390.9

Number of Fisher Scoring iterations: 6
```

When predicting, we pick the method that leads to highest counts (10 min):

```
xnew2$Method <- "10min" # highest duration, highest detection
xnew2$Pred_Rnd <- predict(m5, xnew2, type = "response")

mean(xnew2$Pred_Rnd)
```

```
[1] 0.8329957
```

```r
xnew2 |> ggplot(aes(x = pconif, y = Pred_Rnd)) +
    geom_point() +
    geom_smooth() +
    ylab("Naive density [male individuals/ha]")
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```